

JavaFX TableView Controls Cheat Sheet

Adding buttons, checkboxes, or inputs inside a table row requires a **Custom CellFactory**.

Core Concept: The updateItem Method

To put a control in a cell, you override the updateItem method of a TableCell.

1. **Check if empty:** If the row is empty, clear the graphic (setGraphic(null)).
2. **If not empty:** Configure your control and display it (setGraphic(control)).

1. Action Button (e.g., "Delete" or "Edit")

Scenario: You want a button on every row to perform an action on that specific object.

FXML:

```
<!-- Use Void if the column doesn't map to a specific field, or the ID type -->
<TableColumn fx:id="colAction" text="Actions" />
```

Controller Code:

```
@FXML private TableColumn<Student, Void> colAction;
```

```
@FXML
```

```
public void initialize() {
    // Standard boilerplate for a button column
    colAction.setCellFactory(param -> new TableCell<>() {
        // 1. Create the button ONCE (performance optimization)
        private final Button btn = new Button("Delete");

        {
            // 2. Define the action
            btn.setOnAction(event -> {
                // How to get the object for this row:
                Student data = getTableView().getItems().get(getIndex());
                handleDelete(data);
            });
            btn.getStyleClass().add("danger-button"); // Optional CSS
        }
    });
}
```

```
@Override
```

```
protected void updateItem(Void item, boolean empty) {
```

```

        super.updateItem(item, empty);
        if (empty) {
            setGraphic(null);
        } else {
            setGraphic(btn); // Display the button
        }
    }
});
}

private void handleDelete(Student s) {
    System.out.println("Deleting: " + s.getName());
    // service.delete(s.getId());
    // model.remove(s);
}

```

2. CheckBox (Boolean Field)

Scenario: A boolean field (e.g., isActive, hasPaid) that you want to toggle directly in the table.

Controller Code:

```

@FXML private TableColumn<Student, Boolean> colActive;

@FXML
public void initialize() {
    colActive.setCellValueFactory(new PropertyValueFactory<>("active"));

    colActive.setCellFactory(param -> new TableCell<>() {
        private final CheckBox checkBox = new CheckBox();

        {
            // Update the model when user clicks
            checkBox.setOnAction(event -> {
                Student s = getTableView().getItems().get(getIndex());
                s.setActive(checkBox.isSelected());
                System.out.println("Student " + s.getName() + " active: " + s.isActive());
            });
        }
    });

    @Override
    protected void updateItem(Boolean item, boolean empty) {

```

```

        super.updateItem(item, empty);
        if (empty || item == null) {
            setGraphic(null);
        } else {
            checkBox.setSelected(item); // Set state based on data
            setGraphic(checkBox);
        }
    }
});
}

```

3. ComboBox (Inline Editing)

Scenario: Changing a status (Enum) directly in the table.

Controller Code:

```

@FXML private TableColumn<Student, String> colStatus;

@FXML
public void initialize() {
    colStatus.setCellValueFactory(new PropertyValueFactory<>("status"));

    colStatus.setCellFactory(param -> new TableCell<>() {
        private final ComboBox<String> comboBox = new ComboBox<>();

        {
            comboBox.getItems().addAll("ACTIVE", "INACTIVE", "PENDING");

            // Handle selection change
            comboBox.setOnAction(event -> {
                Student s = getTableView().getItems().get(getIndex());
                s.setStatus(comboBox.getValue());
            });
        }
    });

    @Override
    protected void updateItem(String item, boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null) {
            setGraphic(null);
        } else {

```

```

        comboBox.setValue(item); // Set current value
        setGraphic(comboBox);
    }
}
});
}

```

4. Complex Row (HBox with Multiple Buttons)

Scenario: You need "Edit" AND "Delete" in the same column.

```

colAction.setCellFactory(param -> new TableCell<>() {
    private final Button btnEdit = new Button("Edit");
    private final Button btnDelete = new Button("Delete");
    private final HBox container = new HBox(5, btnEdit, btnDelete); // 5px spacing

    {
        btnEdit.setOnAction(e -> {
            Student s = getTableView().getItems().get(getIndex());
            openEditWindow(s);
        });

        btnDelete.setOnAction(e -> {
            Student s = getTableView().getItems().get(getIndex());
            deleteStudent(s);
        });
    }

    @Override
    protected void updateItem(Void item, boolean empty) {
        super.updateItem(item, empty);
        if (empty) {
            setGraphic(null);
        } else {
            setGraphic(container);
        }
    }
});

```

You should add the business logic (like updating the database or the model object) in the `setOnAction` method, not `updateItem`.

Here is the breakdown of responsibilities:

1. **setOnAction (The Logic):** This only fires when the **user actually clicks**. This is where you update the student object, call a service, or trigger an alert.
2. **updateItem (The Display):** This runs hundreds of times as you **scroll**. Its only job is to look at the data and say "Should this box be checked or not visually?". If you put logic here, it will trigger randomly just by scrolling through the table.

Correct Pattern (from the Cheat Sheet)

Java

```
colActive.setCellFactory(param -> new TableCell<>() {  
  
    // 1. Create Control  
  
    private final CheckBox checkBox = new CheckBox();  
  
    {  
  
        // 2. LOGIC GOES HERE (setOnAction)  
  
        // This only runs when the USER clicks the box  
  
        checkBox.setOnAction(event -> {  
  
            // Get the actual object for the row currently being clicked  
  
            Student s = getTableView().getItems().get(getIndex());  
  
  
            // Execute logic  
  
            s.setActive(checkBox.isSelected());  
        });  
    }  
});
```

```
        System.out.println("Logic executed for: " + s.getName());  
    });  
}
```

@Override

```
protected void updateItem(Boolean item, boolean empty) {  
    // 3. DISPLAY GOES HERE  
  
    super.updateItem(item, empty);  
  
    if (empty || item == null) {  
        setGraphic(null);  
    } else {  
        // Just set the visual state based on data  
        checkBox.setSelected(item);  
        setGraphic(checkBox);  
    }  
}  
};
```

